

TicketApp su Kubernetes

Guida completa: deploy, gestione e monitoraggio

Stack: Angular 17 | Spring Boot 3.2 | PostgreSQL 16 | Kubernetes (Docker Desktop)

1. Cosa abbiamo fatto

Abbiamo migrato l'applicazione TicketApp da Docker Compose a Kubernetes (K8s) locale tramite Docker Desktop. Di seguito il riepilogo completo delle operazioni eseguite.

1.1 Abilitazione di Kubernetes in Docker Desktop

- Aperto Docker Desktop -> Settings -> Kubernetes
- Spuntato 'Enable Kubernetes' -> Apply & Restart
- Atteso circa 2 minuti per l'avvio del cluster
- Verificato con: `kubectl get nodes` (stato atteso: Ready)

1.2 Cambio del contesto kubectl

Era attivo il contesto 'minikube'. Lo abbiamo cambiato a 'docker-desktop':

```
kubectl config use-context docker-desktop
kubectl get nodes
```

1.3 Build delle immagini Docker

Costruite localmente (imagePullPolicy: Never — Kubernetes usa le immagini locali senza bisogno di un registry esterno):

```
docker build -t ticket-backend:1.0.0 ./ticketing-backend
docker build -t ticket-frontend:1.0.0 ./ticketing-frontend
```

1.4 Installazione NGINX Ingress Controller

Necessario per gestire il routing HTTP dall'esterno verso i servizi interni:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/\
controller-v1.10.1/deploy/static/provider/cloud/deploy.yaml

# Attesa che il controller sia pronto:
kubectl wait --namespace ingress-nginx \
--for=condition=ready pod \
```

```
--selector=app.kubernetes.io/component=controller \  
--timeout=120s
```

1.5 Deploy completo dell'applicazione

Applicati tutti i manifest Kubernetes in ordine:

```
kubectl apply -f k8s/namespace.yaml  
kubectl apply -f k8s/secrets.yaml  
kubectl apply -f k8s/configmap.yaml  
kubectl apply -f k8s/postgres/pvc.yaml  
kubectl apply -f k8s/postgres/service.yaml  
kubectl apply -f k8s/postgres/statefulset.yaml  
kubectl apply -f k8s/backend/deployment.yaml  
kubectl apply -f k8s/backend/service.yaml  
kubectl apply -f k8s/frontend/nginx-configmap.yaml  
kubectl apply -f k8s/frontend/deployment.yaml  
kubectl apply -f k8s/frontend/service.yaml  
kubectl apply -f k8s/frontend/ingress.yaml
```

2. Struttura dei file Kubernetes creati

```

ticket-app/
├── k8s/
│   ├── namespace.yaml      -> Namespace 'ticket-app'
│   ├── secrets.yaml        -> Password DB, JWT secret, credenziali mail
│   ├── configmap.yaml      -> URL DB, CORS, porte e configurazione
│   ├── postgres/
│   │   ├── pvc.yaml        -> Disco persistente 2Gi per PostgreSQL
│   │   ├── statefulset.yaml -> Pod PostgreSQL con volume montato
│   │   └── service.yaml     -> Servizio interno (ClusterIP headless)
│   ├── backend/
│   │   ├── deployment.yaml  -> Pod Spring Boot (1 replica, probe TCP)
│   │   └── service.yaml     -> Servizio interno porta 8080
│   └── frontend/
│       ├── nginx-configmap.yaml -> nginx.conf con proxy a backend-service
│       ├── deployment.yaml    -> Pod Angular/Nginx (1 replica)
│       ├── service.yaml       -> Servizio interno porta 80
│       └── ingress.yaml       -> Routing: / -> frontend, /api -> backend

```

Descrizione di ogni file

File	Tipo K8s	Scopo
namespace.yaml	Namespace	Isola tutte le risorse dell'app
secrets.yaml	Secret	Credenziali cifrate (DB, JWT, Mail)
configmap.yaml	ConfigMap	URL DB, CORS, porte, configurazione
postgres/pvc.yaml	PersistentVolumeClaim	Disco da 2 GB per i dati del DB
postgres/statefulset.yaml	StatefulSet	Gestisce il Pod PostgreSQL con stato
postgres/service.yaml	Service (headless)	Permette al backend di parlare col DB
backend/deployment.yaml	Deployment	Mantiene attivo il Pod Spring Boot
backend/service.yaml	Service	Espone il backend sulla porta 8080
frontend/nginx-configmap.yaml	ConfigMap	Override nginx.conf per Kubernetes
frontend/deployment.yaml	Deployment	Mantiene attivo il Pod Angular/Nginx
frontend/service.yaml	Service	Espone il frontend sulla porta 80
frontend/ingress.yaml	Ingress	Entry-point HTTP su localhost

3. Architettura del cluster

```

Browser -> http://localhost
      |
      [Ingress NGINX]
      /      \
  path: /      path: /api/*
      |          |
  [Service: frontend] [Service: backend]
      |          |
  [Pod: Nginx+Angular] [Pod: Spring Boot] x1 (scalabile)
                        |
                        [Service: postgres]
                        |
                        [Pod: PostgreSQL]
                        |
                        [PVC: disco 2 GB]
  
```

Tutti i componenti vivono nel namespace 'ticket-app'. La comunicazione interna avviene tramite i Service (DNS interno Kubernetes). L'unico punto di accesso esterno e' l'Ingress sulla porta 80.

URL dell'applicazione

Apri il browser su: `http://localhost`

Login admin: `admin@company.com / Admin123!`

Login manager: `manager@company.com / Manager123!`

Login utente: `laura@company.com / User123!`

4. Comandi essenziali kubectl

4.1 Stato dei Pod

```

# Elenco pod nel namespace ticket-app
kubectl get pods -n ticket-app

# Con dettagli: IP, nodo, stato esteso
kubectl get pods -n ticket-app -o wide

# Aggiornamento automatico ogni 2 secondi (Ctrl+C per uscire)
kubectl get pods -n ticket-app -w

# Tutti i pod del cluster
kubectl get pods --all-namespaces
  
```

4.2 Log dei servizi

```

# Log del backend (ultime 50 righe)
kubectl logs -n ticket-app deployment/ticket-backend --tail=50

# Log in tempo reale (streaming live)
kubectl logs -n ticket-app deployment/ticket-backend -f
  
```

```
# Log del frontend (nginx)
kubectl logs -n ticket-app deployment/ticket-frontend -f

# Log del database PostgreSQL
kubectl logs -n ticket-app statefulset/postgres -f

# Log degli ultimi 10 minuti
kubectl logs -n ticket-app deployment/ticket-backend --since=10m
```

4.3 Dettagli e debug

```
# Descrizione completa pod (eventi, probe, volumi, errori)
kubectl describe pod -n ticket-app <nome-pod>

# Descrizione di un deployment
kubectl describe deployment -n ticket-app ticket-backend

# Aprire una shell interattiva nel backend
kubectl exec -it -n ticket-app deployment/ticket-backend -- /bin/sh

# Connettersi a PostgreSQL dal terminale
kubectl exec -it -n ticket-app statefulset/postgres \
  -- psql -U postgres -d ticketing_db
```

5. Gestione e operazioni

5.1 Avviare e fermare

```
# Applicare tutti i manifest (avvio o aggiornamento)
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/secrets.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/postgres/
kubectl apply -f k8s/backend/
kubectl apply -f k8s/frontend/

# Eliminare TUTTO (cancella namespace e tutte le risorse)
kubectl delete namespace ticket-app

# Fermare solo il backend (scala a 0 repliche)
kubectl scale deployment ticket-backend --replicas=0 -n ticket-app

# Riportare online
kubectl scale deployment ticket-backend --replicas=1 -n ticket-app
```

5.2 Scaling — aumentare le repliche

```
# Backend a 2 repliche (load balancing automatico)
kubectl scale deployment ticket-backend --replicas=2 -n ticket-app

# Frontend a 2 repliche
kubectl scale deployment ticket-frontend --replicas=2 -n ticket-app

# Verificare lo scaling
kubectl get pods -n ticket-app
```

5.3 Aggiornare l'app — rolling update (zero downtime)

```
# 1. Ricostruisci l'immagine con nuova versione
docker build -t ticket-backend:1.0.1 ./ticketing-backend

# 2. Aggiorna il deployment con la nuova immagine
kubectl set image deployment/ticket-backend \
  backend=ticket-backend:1.0.1 -n ticket-app

# 3. Monitora l'aggiornamento (Kubernetes avvia i nuovi pod
#   prima di spegnere i vecchi: zero downtime)
kubectl rollout status deployment/ticket-backend -n ticket-app

# 4. In caso di problemi: rollback alla versione precedente
kubectl rollout undo deployment/ticket-backend -n ticket-app
```

5.4 Riavvio forzato

```
# Riavvia il backend senza cambiare immagine
kubectl rollout restart deployment/ticket-backend -n ticket-app
```

```
# Riavvia il frontend
kubectl rollout restart deployment/ticket-frontend -n ticket-app

# Riavvia tutti i deployment del namespace
kubectl rollout restart deployment -n ticket-app
```

6. Monitoraggio e risorse

6.1 Consumo CPU e RAM

```
# Consumo risorse dei pod (richiede metrics-server)
kubectl top pods -n ticket-app

# Consumo del nodo
kubectl top nodes

# Installare metrics-server se non presente
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/\
releases/latest/download/components.yaml
```

6.2 Panoramica completa del namespace

```
# Tutto in un colpo solo (pod, deployment, service, ingress)
kubectl get all -n ticket-app

# Solo i servizi
kubectl get services -n ticket-app

# Ingress con URL e path
kubectl get ingress -n ticket-app

# ConfigMap e Secrets
kubectl get configmap -n ticket-app
kubectl get secrets -n ticket-app
```

6.3 Contesti kubectl disponibili

```
# Lista tutti i contesti (cluster configurati)
kubectl config get-contexts

# Contesto attivo
kubectl config current-context

# Cambiare contesto
kubectl config use-context docker-desktop
kubectl config use-context minikube
```

6.4 Eventi del cluster

```
# Tutti gli eventi del namespace ordinati per data
kubectl get events -n ticket-app --sort-by=.lastTimestamp

# Solo gli warning e gli errori
kubectl get events -n ticket-app --field-selector type=Warning
```

6.5 Gestire Secrets e ConfigMap

```
# Vedere il contenuto di un secret (valori in base64)
```



```
kubect1 get secret ticket-secrets -n ticket-app -o yaml

# Decodificare un valore specifico
kubect1 get secret ticket-secrets -n ticket-app \
  -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d

# Modificare un ConfigMap (apre editor)
kubect1 edit configmap ticket-config -n ticket-app

# Applicare dopo la modifica
kubect1 rollout restart deployment/ticket-backend -n ticket-app
```

7. Troubleshooting — problemi comuni

Problema: Pod in stato: CrashLoopBackOff

Il container crasha subito dopo l'avvio. Kubernetes riprova in loop.

```
kubectl logs -n ticket-app <nome-pod> --previous
kubectl describe pod -n ticket-app <nome-pod>
# Leggi la sezione 'Events' in fondo all'output
```

Problema: Pod in stato: Pending

Il pod non riesce ad essere schedulato su nessun nodo.

```
kubectl describe pod -n ticket-app <nome-pod>
# Cause comuni: risorse insufficienti (RAM/CPU), PVC non disponibile
```

Problema: Pod in stato: ImagePullBackOff

Kubernetes non trova l'immagine Docker specificata.

```
# Verifica che le immagini siano state costruite localmente
docker images | grep ticket
# Verifica che deployment.yaml abbia: imagePullPolicy: Never
```

Problema: Backend non raggiunge il database

Errore di connessione a PostgreSQL all'avvio di Spring Boot.

```
# Verifica che il pod postgres sia Running
kubectl get pods -n ticket-app
# Testa la connessione TCP dall'interno del backend
kubectl exec -it -n ticket-app deployment/ticket-backend -- \
/bin/sh -c 'nc -zv postgres-service 5432'
```

Problema: Ingress non risponde su http://localhost

Il traffico HTTP non arriva ai servizi.

```
# Verifica stato Ingress Controller
kubectl get pods -n ingress-nginx
# Verifica che ADDRESS sia 'localhost'
kubectl get ingress -n ticket-app
# Ricontrolla i log dell'ingress controller
kubectl logs -n ingress-nginx deployment/ingress-nginx-controller
```

Problema: Dati del database persi dopo riavvio

Il PVC non e' stato creato correttamente.

```
# Verifica che il PVC sia Bound
kubectl get pvc -n ticket-app
# Deve mostrare STATUS: Bound
```

8. Cheatsheet rapido — tutti i comandi in una pagina

Operazione	Comando
Stato pod	<code>kubectl get pods -n ticket-app</code>
Stato pod (live)	<code>kubectl get pods -n ticket-app -w</code>
Tutto il namespace	<code>kubectl get all -n ticket-app</code>
Log backend live	<code>kubectl logs -n ticket-app deployment/ticket-backend -f</code>
Log frontend live	<code>kubectl logs -n ticket-app deployment/ticket-frontend -f</code>
Log database live	<code>kubectl logs -n ticket-app statefulset/postgres -f</code>
Log ultimi 10 min	<code>kubectl logs -n ticket-app deployment/ticket-backend --since=10m</code>
Dettagli pod	<code>kubectl describe pod -n ticket-app <nome-pod></code>
Shell nel backend	<code>kubectl exec -it -n ticket-app deployment/ticket-backend -- /bin/sh</code>
Shell in PostgreSQL	<code>kubectl exec -it -n ticket-app statefulset/postgres -- psql -U postgres -d ticketing_db</code>
Scala backend 2x	<code>kubectl scale deployment ticket-backend --replicas=2 -n ticket-app</code>
Riavvia backend	<code>kubectl rollout restart deployment/ticket-backend -n ticket-app</code>
Riavvia frontend	<code>kubectl rollout restart deployment/ticket-frontend -n ticket-app</code>
Rollback backend	<code>kubectl rollout undo deployment/ticket-backend -n ticket-app</code>
Stato rolling update	<code>kubectl rollout status deployment/ticket-backend -n ticket-app</code>
Ingress e routing	<code>kubectl get ingress -n ticket-app</code>
Servizi interni	<code>kubectl get services -n ticket-app</code>
ConfigMap	<code>kubectl get configmap -n ticket-app</code>
Secrets	<code>kubectl get secrets -n ticket-app</code>
Eventi (ordinati)	<code>kubectl get events -n ticket-app --sort-by=.lastTimestamp</code>
Solo warning/errori	<code>kubectl get events -n ticket-app --field-selector type=Warning</code>
Contesti disponibili	<code>kubectl config get-contexts</code>
Contesto attivo	<code>kubectl config current-context</code>
Cambia contesto	<code>kubectl config use-context docker-desktop</code>
Applica tutti manifest	<code>kubectl apply -f k8s/</code>
ELIMINA TUTTO	<code>kubectl delete namespace ticket-app</code>

Note importanti

- Le immagini Docker sono locali: se ricostruisci devi fare 'kubectl rollout restart'
- Per un deploy su cloud (GKE, EKS, AKS) pubblica le immagini su un registry (Docker Hub, ECR...)
- Il namespace 'ticket-app' contiene tutte le risorse: eliminarlo rimuove tutto
- I dati del DB sopravvivono ai riavvii grazie al PVC (PersistentVolumeClaim)